CSCE883 Machine Learning **Name:** *Taylor Belcher*
Homework 3
10/11/2026

---

1. *(Question 1A.)* We have a linear network with L layers with the initial input $h_0 = x$ and at each subsesquent layer l (where $1 \leq l \leq L$), $h_l = \sigma(w_l h_{l-1})$ and $\sigma(z) = \dfrac{1}{1+e^{-z}}$. $|w_l| \leq 1$ and there are no biases.

   (i) We want to estimate an upper bound on $\left|\dfrac{\partial h_L}{\partial x}\right|$ in terms of L. We have that

   $$\begin{aligned} h_L &= \sigma(w_L h_{L-1}) \\ &= \sigma\big(w_L \sigma\big(w_{L-1}\sigma(\cdots\sigma(w_1 x))\big)\big). \end{aligned}$$

   At any given level $l \geq 2$, let $g_l(x)$ denote the remaining nested expression, including the weight w_l, all the way down to the input x:

   $$g_l(x) = w_l \sigma\big(w_{l-1}\sigma(\cdots\sigma(w_1 x))\big), \qquad g_1(x) = w_1 x.$$

   Thus $h_l = \sigma(g_l(x))$. By the chain rule, the partial derivative of h_L with respect to x can be written as

   $$\begin{aligned} \left|\frac{\partial h_L}{\partial x}\right| &= |\sigma'(g_L(x))\, g_L'(x)| \\ &= |\sigma'(g_L(x))|\ \left|w_L\, \sigma'(g_{L-1}(x))\, g_{L-1}'(x)\right| \\ &= |\sigma'(g_L(x))|\ |w_L\, \sigma'(g_{L-1}(x))| \\ &\quad \cdot |w_{L-1}\, \sigma'(g_{L-2}(x))| \cdots |w_2\, \sigma'(g_1(x))|\ |w_1| \\ &= \left(\prod_{l=1}^{L} |w_l|\right)\left(\prod_{l=1}^{L} |\sigma'(g_l(x))|\right) \\ &\leq 1 \cdot \prod_{l=1}^{L} |\sigma'(g_l(x))|\,. \end{aligned}$$

   The inequality follows from $|w_l| \leq 1$ for every layer, so $\prod_{l=1}^{L} |w_l| \leq 1$. Now the derivative of the sigmoid activation function is

   $$\sigma'(z) = -(1+e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}.$$

   To bound this derivative, observe that

   $$1 - \sigma(z) = 1 - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}.$$

Thus we can rewrite the derivative as

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

Since $0 < \sigma(z) < 1$, this derivative is positive. Completing the square gives

$$|\sigma'(z)| = \sigma(z)\big(1-\sigma(z)\big) = \frac{1}{4} - \left(\sigma(z) - \frac{1}{2}\right)^2 \leq \frac{1}{4}.$$

Applying this bound to each of the L factors in our product yields

$$\begin{aligned}\left|\frac{\partial h_L}{\partial x}\right| &\leq \prod_{l=1}^{L} |\sigma'(g_l(x))| \\ &\leq \prod_{l=1}^{L} \frac{1}{4} \\ &= \boxed{\left(\frac{1}{4}\right)^L = 4^{-L}}.\end{aligned}$$

Therefore the magnitude of the input derivative is bounded above by 4^{-L}, which decreases exponentially with the number of layers. Increasing the number of layers will decrease the gradient exponentially, making it so that earlier layers will take much longer to train.

(ii) If the weights were not bounded above by 1, then we could counter-balance the derivative of the sigmoid function with a larger product of weights, but this is not a guarantee as the weights would also increase the input to the sigmoid functions and the larger the input, the flatter the curve on the sigmoid gets, which means the derivative would also be getting smaller as it moves into the saturation region. So increasing the weights can help with a vanishing gradient, but it is not a guaranteed solution. More precisely, the weight bound matters because each layer contributes both a weight and a sigmoid derivative:

$$\left|\frac{\partial h_l}{\partial h_{l-1}}\right| = |w_l \sigma'(w_l h_{l-1})| \leq \frac{|w_l|}{4}.$$

With $|w_l| \leq 1$, each factor is at most $1/4$, giving the exponential bound in part (i). Without this weight bound, larger weights could offset small sigmoid derivatives, so small sigmoid derivatives alone do not prove that every deep network must have vanishing gradients. However, larger weights can also move the sigmoid inputs into regions where its derivative is small. Increasing the weights is therefore not a guaranteed solution.

A vector residual block adds its input back to a learned transformation of that input:

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F_l(\mathbf{h}_l), \qquad F_l : \mathbb{R}^d \to \mathbb{R}^d.$$

Its Jacobian contains the partial derivative of each output component with respect to each input component. Writing the ith output component as $h_{l+1,i} = h_{l,i} + F_{l,i}(\mathbf{h}_l)$ gives

$$\begin{aligned}[J_l]_{ij} &= \frac{\partial h_{l+1,i}}{\partial h_{l,j}} \\ &= \frac{\partial h_{l,i}}{\partial h_{l,j}} + \frac{\partial F_{l,i}(\mathbf{h}_l)}{\partial h_{l,j}} \\ &= \delta_{ij} + [J_{F_l}(\mathbf{h}_l)]_{ij},\end{aligned}$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, is Kroenecker's delta. These entries form the identity matrix, so

$$\boxed{J_l = I + J_{F_l}(\mathbf{h}_l)}.$$

For a scalar loss $\mathcal{L}$, the chain rule propagates its gradient backward through the block as

$$\nabla_{\mathbf{h}_l}\mathcal{L} = \big(I + J_{F_l}(\mathbf{h}_l)\big)^{\mathsf{T}} \nabla_{\mathbf{h}_{l+1}}\mathcal{L}.$$

The identity term provides a direct contribution from the next layer's gradient. When J_{F_l} is small, the block's Jacobian is close to I, which helps preserve gradients. This does not guarantee successful training: for example, $J_{F_l} = -I$ would cancel the identity term entirely, and products of Jacobians across many blocks can still shrink or amplify gradients.

2. *(Question 1B.)*

The input is $3 \times 24 \times 24$ in channel–height–width order. Convolutions have stride 1, no padding, dilation 1, and one bias per output channel. Pooling uses a 2×2 window with stride 2, no padding, and floor rounding.

(i) For either spatial dimension n, the output-size formula is

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - d(k-1) - 1}{s} \right\rfloor + 1,$$

where k is the kernel size, p is padding, d is dilation, and s is stride. Here $p = 0$ and $d = 1$, so this becomes $n_{\text{out}} = \lfloor (n_{\text{in}} - k)/s \rfloor + 1$. Since height and width are equal, the same calculation applies to both:

Layer	Spatial calculation	Output dimension
Conv 1	$\lfloor (24-3)/1 \rfloor + 1 = 22$	$12 \times 22 \times 22$
Pool 1	$\lfloor (22-2)/2 \rfloor + 1 = 11$	$12 \times 11 \times 11$
Conv 2	$\lfloor (11-3)/1 \rfloor + 1 = 9$	$128 \times 9 \times 9$
Pool 2	$\lfloor (9-2)/2 \rfloor + 1 = 4$	$128 \times 4 \times 4$

A convolution's channel count is its number of output filters, while pooling preserves the channel count. In Pool 2, floor rounding gives 4, so the last row and column of the 9×9 input are not covered by a pooling window.

(ii) Each convolutional output filter spans all input channels. Therefore a $k \times k$ filter has $C_{\text{in}}k^2$ weights and one bias, giving

$$N_{\text{conv}} = C_{\text{out}} \cdot (C_{\text{in}}k^2 + 1).$$

For Conv 1, each output filter has $3 \cdot 3 \cdot 3 = 27$ weights and one bias. With 12 output filters, the total is

$$N_{\text{Conv 1}} = 12(27 + 1) = \boxed{336} \quad (324 \text{ weights and } 12 \text{ biases}).$$

Conv 2 receives 12 channels, so each output filter has $12 \cdot 3 \cdot 3 = 108$ weights and one bias. With 128 output filters,

$$N_{\text{Conv 2}} = 128(108 + 1) = \boxed{13{,}952} \quad (13{,}824 \text{ weights and } 128 \text{ biases}).$$

Max pooling only selects the maximum within each window and has no trainable weights or biases. Thus Pool 1 and Pool 2 each have $\boxed{0}$ trainable parameters.

(iii) After Pool 2, flattening the $128 \times 4 \times 4$ tensor gives

$$n_{\text{flat}} = 128 \cdot 4 \cdot 4 = \boxed{2{,}048}$$

features. A fully connected layer with 10 outputs has one weight for each input–output pair and one bias per output, so it adds

$$N_{\text{FC}} = 2{,}048 \cdot 10 + 10 = \boxed{20{,}490}$$

trainable parameters. Flattening itself has no trainable parameters.

In a convolution, the same filter weights and bias are reused at every spatial location. Thus its parameter count depends on the kernel size and channel counts, rather than the number of output locations. A fully connected mapping learns separate weights for every input–output pair. For example, a fully connected mapping from the original $3 \cdot 24 \cdot 24 = 1{,}728$ inputs to the $12 \cdot 22 \cdot 22 = 5{,}808$ outputs of Conv 1 would require

$$1{,}728 \cdot 5{,}808 + 5{,}808 = 10{,}042{,}032$$

parameters, compared with Conv 1's 336. This reduction comes from both local connectivity (each output uses a 3×3 spatial patch) and weight sharing (all locations use the same filters).